

LLM 预训练之 数据解析 Data Parsing

从 WARC / HTML / PDF / Code 到 Canonical Document 的系统工程

Know-Why × Know-How × Parser Evaluation × Production Blueprint

研究版本：2026-08

基于公开论文、官方数据管线与工具文档

执行摘要：Parsing 不是“格式转换”，而是训练分布的第一道建模

核心判断：数据解析的目标不是尽可能抽出更多字符，而是把异构原始内容映射为“语义正确、结构尽量保真、可追溯、可比较、可下游过滤”的 Canonical Document。解析器会改变 token yield、内容覆盖和结构保真，因此它本身就是预训练数据配方的一部分。

FineWeb 的受控 ablation 给出了非常直接的证据：从 Common Crawl WARC 用 Trafilatura 重新抽取文本，比直接使用 Common Crawl 的 WET 文本训练出的模型更好；WET 版本虽然约多 25% token，但增加的很多是 boilerplate、导航等低价值内容。[1]

2026 年的 Beyond a Single Extractor 进一步指出，“固定一个 HTML extractor”本身可能造成数据覆盖损失：不同 extractor 经过相同过滤后留下的页面集合差异很大；把多个 extractor 的结果做 union，DCLM-Baseline token yield 可提高最多 71% 而保持通用基准表现，并且表格/代码等结构内容的 extractor 选择可带来明显的下游差异。[2]

因此，现代预训练数据解析应从“单解析器流水线”升级为“路由 + 结构保留 + 多解析器候选 + 质量判定 + parser ablation”的数据系统。PDF/Office 等复杂文档还需要布局、OCR、表格、公式和 reading order 等模块；Docling、MinerU、GROBID 等工具代表了不同专长路线。[6][7][8]

问题	传统做法	更合理的预训练视角
HTML 怎么转文本？	选一个 extractor 全量跑	把 extractor 当作数据超参数；按页面类型路由，必要时 union/ensemble
文本越多越好吗？	最大化字符/token yield	最大化“有效 token yield”：正文覆盖 - boilerplate - 结构破坏
PDF 怎么抽？	pdftotext/OCR 一把梭	先判断 native/scanned/complex layout，再路由到 text/layout/OCR/VLM parser
输出什么？	txt	Canonical JSON: text + blocks + metadata + provenance + parser diagnostics
如何评估？	人工看几十个样本	抽取准确度 + 结构指标 + 失败率 + 成本 + 训练小模型 ablation

1. 模块边界：Parsing、Cleaning、Dedup、Tokenization 分别负责什么

阶段	输入	主要职责	不应该偷偷做什么	输出
Acquisition	URL/WARC/PDF/Repo	获得原始字节和来源信息	不应丢 provenance	raw object + metadata
Parsing / Extraction	raw bytes / markup / file tree	解码、结构识别、正文抽取、reading order、元素恢复	不要把“质量判断”与“内容抽取”混成不可追踪的一步	canonical document
Normalization	Unicode/text blocks	规范编码、空白、可逆或可记录的格式归一	不要无记录地改语义	normalized canonical document
Filtering / Cleaning	canonical document	去低质、广告、垃圾、非法/不需要内容	不要依赖 parser 隐式丢数据	kept/dropped + reasons
Dedup	filtered documents	识别 exact/fuzzy/semantic 重复	不要误把解析差异当“独特内容”	deduped corpus
Tokenization / Packing	final text/structure serialization	转 token、拼接训练序列	不要承担正文抽取	training batches

边界为什么重要：如果 parser 在内部把某些块“悄悄删除”，后续你无法区分“原文不存在”与“解析失败”；无法做 parser 对比、回放、质量审计和数据版本迁移。最好的实践是保留 raw reference、parser version、block-level provenance 与 drop reason。

2. 统一抽象：Canonical Document 是数据解析层的真正产品

无论输入是网页、PDF、论文、Office 文档还是代码仓库，解析层最好输出统一但可扩展的文档对象。这样后续 filtering、dedup、quality scoring、mixture 都可以在统一接口上运行。Dolma 的统一 document 格式也强调 id/text/source/metadata 和可追溯性；Docling 则进一步用 body/furniture/tree 表达内容层级与 reading order。[9][7]

```
{
  "id": "stable_source_id",
  "source": "commoncrawl|paper|repo|book|...",
  "raw_ref": {"uri": "...", "offset": 123, "digest": "..."},
  "text": "serialized training text",
  "blocks": [
    {"type": "heading", "text": "...", "order": 0, "bbox": null},
    {"type": "paragraph", "text": "...", "order": 1},
    {"type": "code", "text": "...", "lang": "python", "order": 2},
    {"type": "table", "html": "...", "text": "...", "order": 3}
  ],
  "metadata": {"title": "...", "author": "...", "url": "...", "created": "..."},
  "parser": {"name": "...", "version": "...", "mode": "...", "confidence": 0.97},
  "diagnostics": {"decode_errors": 0, "boilerplate_ratio": 0.12, "ocr_used": false}
}
```

字段	Know-why	工程建议
id/raw_ref	支持去重、回溯、合规删除、重跑 parser	source-specific stable id + hash + storage offset
text	训练器最常消费的最终序列	应由 blocks 可确定地序列化得到，避免两套事实来源
blocks	保留标题/段落/列表/代码/表格/公式结构	尽量带 type/order/bbox/source span
metadata	支持质量模型、采样、归因和过滤	标题/时间/语言/域名/license 等不要混进正文
parser info	支持版本化与 A/B	parser 名称、参数、commit/version、fallback 路径
diagnostics	支持监控和路由	解码失败、OCR、布局置信、token/char ratio、异常长度等

3. Web Archive 入口：WARC、WAT、WET 的角色

Common Crawl 的 WARC 保存原始抓取数据与 HTTP 响应；WAT 保存从 WARC 计算出的元数据；WET 保存预抽取的纯文本。[3] 2026 年 6 月的 CC-MAIN-2026-25 包含约 21 亿网页，压缩 WARC 约 82.68 TiB、WET 约 5.69 TiB，直观体现了“直接用 WET 更便宜，但 WARC 信息更完整”的成本差异。[4]

格式	包含什么	优点	缺点	适用场景
WARC	HTTP headers + 原始 payload/HTML + crawl metadata	信息最完整；可重跑任意 extractor；保留结构和元数据	存储/IO/CPU 成本最高	高质量预训练数据、需要 parser 实验/重跑
WAT	解析后的 header/link/HTML metadata JSON	适合结构分析、链接图、元数据过滤	没有完整正文	预过滤、URL/metadata 分析
WET	Common Crawl 已抽取 plain text	体积小、读取快、成本低	正文抽取质量不可控；结构丢失；boilerplate 可能更多	低预算 baseline、快速实验

FineWeb 的关键证据：在 2019-18 crawl 的实验里，WET 版本约 254B token，而 WARC+Trafilatura 约 200B token；尽管 WET 数据约大 25%，对应模型表现更差，作者通过样本检查将额外 token 主要归因为 boilerplate/导航噪声。[1]

4. Web Parsing Step 0：字节、HTTP、Content-Type 与编码

WARC record / HTTP response bytes
Content-Type / Content-Encoding / charset hints
decompression + byte decoding
HTML parser / DOM construction
text & structure extraction

解析失败经常不是 extractor 算法本身，而是前面的字节层问题：错误 Content-Type、压缩流损坏、页面声明与真实 charset 冲突、混合编码、非法 Unicode、截断响应。WHATWG Encoding Standard 维护 Web 解码规则；Unicode UAX #15 定义 NFC/NFD/NFKC/NFKD 等规范化形式。[10][11]

问题	症状	建议
Content-Encoding 错误	gzip/br 解压失败或乱码	记录 HTTP header；失败时有受控 fallback；不要吞异常
charset 错误	中文/欧洲语言 mojibake	按 Web 解码规则组合 BOM/header/meta/检测器；保留原始 bytes hash
截断网页	DOM 尾部缺失、脚本/表格不闭合	容错 HTML parser + truncated flag；Common Crawl 对异常大流有抓取截断上限
Unicode 非规范形式	相同文本 hash 不一致	文本阶段通常使用 NFC；NFKC 需谨慎，可能折叠兼容字符语义
控制字符/零宽字符	tokenization 异常	可统计并在 normalization 阶段处理；不要在 parser 阶段不可逆删除

5. HTML DOM 解析：先恢复树，再谈“正文”

Web HTML 经常是无效、缺标签、深度异常或动态页面。生产 parser 需要浏览器级容错或高性能 DOM parser。Resiliparse 使用 Lexbor DOM，并提供面向大规模 web archive 的 FastWARC、编码/content-type、HTML parsing 和 process guard 等组件。[5]

DOM 层动作	为什么	典型风险
移除 script/style/template 等不可见内容	减少非文本噪声	noscript 可能包含有价值替代内容
识别 block-level 元素	恢复段落/列表/标题边界	错误合并会破坏句子和表格
识别 hidden 元素	避免菜单/弹窗/不可见文本	CSS/JS 动态可见性难以只靠静态 HTML 判断
保留 pre/code	代码语义依赖空白与换行	通用 whitespace collapse 会破坏代码
解析 link/alt/meta	可用于结构、质量、图片描述	href/alt 是否进入训练正文需单独决策
处理 malformed DOM	真实 Web 不满足规范	严格 XML parser 会大量失败

6. Main Content Extraction：Precision 与 Recall 的核心权衡

HTML→Text 不是“取所有 text node”。页面里有导航、footer、推荐、cookie notice、评论、侧栏、广告、目录等。主要内容抽取器通常通过 DOM、文本密度、链接密度、标签/类名、语义规则等估计“正文”。Trafilatura 明确提供 favor_precision/favor_recall 取舍；Resiliparse 也提供 main_content 模式和可选的 comments/post_meta/hidden_elements 等控制。[1][5][12]

目标	高 Precision 配置	高 Recall 配置	预训练影响
正文纯度	更积极删除边缘块	保留更多可能正文	precision 高可提高单位 token 质量，但可能丢长尾结构
评论/UGC	经常删除	可保留	UGC 能增加口语和问答分布，但噪声更大

目标	高 Precision 配置	高 Recall 配置	预训练影响
链接/alt	通常不进入正文	可序列化	结构信息有价值，但 URL 会产生 token 噪声
列表/目录	易被当 boilerplate	保留更多	技术文档/百科中列表可能是高价值内容
表格/代码	传统 extractor 常损伤	结构感知 extractor 重点保留	直接影响 code/table 能力

关键工程原则：不要把“更少文本”自动理解为“更高质量”。解析器的 precision/recall 需要在训练目标、页面类型和后续过滤能力之间共同优化。

7. HTML Extractor 路线图：规则式 → 多解析器 → 模型式

路线	代表	核心机制	优势	短板	适合
规则/启发式 main-content	Trafilatura	DOM + 内容/密度/规则；可调 precision/recall	成熟、CPU 友好、可解释	结构化元素易损伤；单一规则难覆盖所有页面	大规模通用 Web baseline
高性能规则式	Resiliparse	高性能 DOM + rule-based HTML2Text/main content	吞吐高、适合 WARC 流处理	语义能力有限	超大规模 CPU pipeline
多 extractor union/ensemble	Beyond a Single Extractor	并行多 extractor，基于过滤/去重形成联合集合	提升覆盖；避免单 parser 偏差	成本增加；要解决同页多版本去重/选择	长 horizon、大数据池
轻量模型式	Dripper	HTML 简化 + block sequence classification + controlled decoding	语义理解更强；减少 hallucination	模型推理成本	对结构要求高的 Web
模型式结构解析	MinerU-HTML/AICC	语义块分类 + Markdown 结构恢复	代码/公式/表格保留更好	系统复杂，需 GPU/模型服务	高价值/结构丰富页面

2026 年的 Beyond a Single Extractor 观察到，不同 extractor 对通用语言基准可能相近，但过滤后保留下来的网页集合差异显著；union 可将 DCLM-Baseline token yield 最多提高 71%，同时保持 benchmark performance。对结构化内容，选择不同 extractor 可在 WikiTQ 上产生最高约 10 个百分点差异，在 HumanEval 上约 3 个百分点差异。[2]

AICC/MinerU-HTML 则把正文提取重构为模型化序列标注和结构化格式恢复；其论文报告在 MainWebBench 上显著高于 Trafilatura，并在相同后续过滤下通过预训练实验观察到下游收益。这说明“解析质量”本身可以成为能力提升来源，而不只是过滤前的固定 ETL。[13]

8. 结构保留：为什么 Plain Text 已经不够

结构元素	错误解析示例	潜在损伤	建议序列化
Heading	标题和正文粘成一行	文档层级、主题切换信号丢失	Markdown heading / typed block
List	项目符号全被抹掉	步骤、枚举、并列关系丢失	保留 list item + nesting
Code	缩进压平、HTML entity 未解码	语法和程序语义受损	fenced code + language metadata
Table	按 DOM 顺序变成长字符流	行列关系丢失	HTML/Markdown table + fallback text
Formula	MathML/LaTeX 被删	数学符号知识损失	优先 LaTeX/MathML canonicalization
Quote	正文/引用无法区分	语义角色混淆	blockquote type
Link	anchor 与目标彻底分离	知识图关系损失	metadata 中保留 href，正文按策略保留 anchor

Why：预训练本质上是从序列中学习统计结构。解析器如果提前把表格、代码、标题层级压平成“无结构字符流”，后续模型再大也无法从不存在的结构信号中恢复原信息。

9. PDF 解析：它不是文本格式，而是页面绘制格式

PDF 的核心难点是“视觉位置 ≠ 逻辑阅读顺序”。同一个页面可能包含多栏、浮动图、脚注、页眉页脚、公式、表格、旋转文本；扫描 PDF 甚至没有文本层。因而一个可靠的 PDF parser 通常需要：页面分类 → native text/OCR → layout detection → block recognition → reading order → table/formula → cross-page reconstruction。

PDF bytes
Page classification: native / scanned / hybrid / complex
Native text extraction OR OCR/VLM recognition
Layout blocks: title / text / list / table / figure / formula
Reading-order reconstruction
Table + Formula + Figure caption handling
Cross-page merge / headers-footers / hierarchy
Canonical blocks + Markdown/JSON

PDF 类型	首选策略	原因	Fallback
Born-digital 单栏文本	native text + coordinates	最快、字符最准确	layout parser
Born-digital 多栏/学术	native text + layout/reading order	仅按 PDF text stream 容易乱序	GROBID/Docling/MinerU
扫描 PDF	OCR/VLM	没有可靠 text layer	多 OCR ensemble / manual QA subset
复杂表格/公式	layout + specialized recognizer/VLM	纯 OCR 不能恢复二维结构	table/formula model
混合 PDF	逐页路由	部分页有 text layer、部分扫描	page-level classifier

10. PDF 工具路线对比：Docling、MinerU、GROBID

工具	定位	强项	输出/结构	适合预训练场景	注意
Docling	通用文档 conversion	布局分析、表格结构、统一 DoclingDocument；支持 body/furniture/tree reading order	结构化 document + Markdown/JSON 等	通用 PDF/Office ingestion，重视结构与统一 schema	需要按规模评估吞吐和模型成本
MinerU	复杂文档高精度 parsing	多栏、公式、表格、扫描、reading order、header/footer、跨页结构；2026 继续强化	Markdown/JSON/中间结构	科学文献、复杂 PDF、多语言 OCR	版本演进快，需固定模型/参数做可复现版本
GROBID	科学论文专用	题名/作者/摘要/章节/参考文献/引用等学术结构	TEI XML	学术论文语料、S2ORC/peS2o 类管线	领域专用；对任意商业 PDF 不等于通用 parser

Docling 的 document model 将 texts/tables/pictures/key-value items 与 body/furniture/groups 分离，并用树结构表达阅读顺序；这是很适合预训练数据层的思路，因为“页眉页脚等 furniture”可以保留但不必混入正文。[7]

MinerU 官方说明强调 PDF/图片/Office 到 Markdown/JSON、去页眉页脚、恢复人类阅读顺序、公式转 LaTeX、表格转 HTML、扫描文档 OCR 等；MinerU2.5/2.5-Pro 的研究继续把复杂 document parsing 的提升集中在粗到细识别和数据工程上。[6][14]

GROBID 专注技术与学术 PDF，输出 TEI XML，并支持 header、bibliographic references 等结构化抽取；peS2o 的 S2ORC-derived 数据就使用 GROBID 提取 section headers 与 paragraphs，并移除 figures/tables/references 等非目标内容。[8][15]

11. PDF Parser 如何评估：不要只算 OCR 字符正确率

OmniDocBench 的设计说明了现代 document parsing 的评估维度：除了文本识别，还需要 layout、table、formula、reading order。其最新公开版本覆盖多种文档/布局/语言，并使用 Normalized Edit Distance、TEDS、公式指标等。[16]

维度	指标示例	为什么重要	预训练特有观察
Text OCR	CER/WER/Edit Distance	字符是否正确	文字错误会直接制造错误 token
Layout Detection	mAP/mAR	元素类型与位置是否正确	决定标题/正文/脚注/表格是否混合
Reading Order	order edit / sequence accuracy	块顺序是否符合阅读逻辑	乱序会制造 “错误因果/篇章结构”
Table Structure	TEDS / cell matching	二维关系是否保留	影响表格 QA、数字推理
Formula	LaTeX edit / render-based metric	数学符号是否正确	一个符号错误可改变公式语义
Yield	valid chars/tokens per page	parser 取回多少有效内容	不能只优化准确率导致过度丢失
Failure Rate	timeout/OOM/empty/garbled	规模化稳定性	0.1% failure 在十亿页面上就是百万级
Cost	CPU-sec/GPU-sec/GB	决定是否可扩展到训练规模	高质量 parser 可采用 selective routing
Downstream Ablation	小模型 loss/benchmarks	最终是否改善模型	最终裁决指标

12. 学术论文解析：S2ORC / peS2o 给出的工程模板

学术语料的难点不是只有 OCR，而是“结构化语义”：title、abstract、section、paragraph、citation、bibliography、figure/table mentions 的对象关系。S2ORC 建立了 PDF/LaTeX/JATS→统一 JSON 的路线；其 doc2json 工具使用 GROBID 将 PDF 转为 TEI XML，再映射为 S2ORC JSON。[17]

阶段	操作	Know-why
Source resolution	论文 ID + metadata + PDF/LaTeX/JATS source	metadata 与正文解析分离，便于更新与纠错
Format-specific parser	PDF→GROBID；LaTeX/JATS 专用 parser	不同源格式的信息损失不同，不能强行统一入口
Object linking	citation mention → bibliography entry	保留学术知识图结构
Content policy	决定 figures/tables/references 是否进入文本	peS2o 的目标是高效预训练，因此只保留特定文本结构
Language/quality	language ID、length、unigram 等过滤	解析与质量过滤是连续但应可审计的阶段

13. Code Repository Parsing：代码不是“一堆文件文本”

代码预训练应保留 repo/file/path/language/license 等上下文。The Stack v2 从 Software Heritage 图数据恢复目录树，选择最新主分支 revision、按内容 hash 去 repo 重复、下载文件内容，并使用语言分类器而不是仅靠扩展名识别语言。[18]

解析对象	关键字段	解析/规范化逻辑	为什么
Repository	repo id, origin, revision, main branch	固定 snapshot，保留 commit/provenance	保证可复现与 repo-level 关系
Directory Tree	path, parent, depth	重建完整路径与目录层级	repo-level completion 依赖跨文件关系
File	path, bytes, size, mime/lang	二进制/生成文件识别；受控大小限制	避免 vendored/binary/超大文件污染
Language	language + confidence	classifier/linguist 类工具	扩展名不足以识别 notebook/template/无扩展文件
License	repo/file-level license	扫描 LICENSE/README 等并传播到文件范围	治理与可用范围决定
Notebook	cells + cell type + outputs	按 cell 序列化，保留 code/markdown 关系	notebook 是代码+解释的天然混合文档
Issue/PR	title/body/comments/diff metadata	结构化线程，不要简单 concat	对 coding assistant 的问题解决能力有价值

一个常见错误：把整个 repo 解压后按“所有文本文件”读取，会丢失版本、目录关系、license、语言和来源，并让 vendored/generated 文件与核心源码同权进入语料。预训练 code parser 应把 repository graph 当作一等结构。

14. Office / 表格 / 幻灯片：优先原生解析，而不是先转 PDF

格式	优先入口	要保留的结构	不要做的事
DOCX	OOXML/native parser	heading、paragraph、list、table、footnote、hyperlink、section	无必要先转 PDF；会把逻辑结构降维成页面
PPTX	OOXML/native parser	slide order、title、textbox、notes、table、shape relationship	把每页 OCR 成无结构文本
XLSX	OOXML/native parser	sheet、cell、row/col、formula、merged cell、header	只导出“看起来的文本”丢公式/坐标
EPUB/HTML Book	DOM/EPUB spine	chapter order、heading、paragraph、footnote	按 zip 内文件字典序拼接

这条原则与现代 MinerU/Docling 的多格式路线一致：当原始格式本身携带逻辑结构时，优先 native parser；“先渲染再 OCR”应是处理扫描或不可解析内容的 fallback，而不是默认路径。[6][7]

15. Parsing 后的文本规范化：什么该改，什么不该改

操作	推荐位置	通常建议	风险
HTML entity decode	parser	应做	需注意代码/文本边界
Unicode NFC	normalization	通常推荐	对部分特殊任务需保留原形式
NFKC	normalization	谨慎、按语言/任务	会折叠兼容字符，可能改变语义
Whitespace collapse	block-aware normalization	普通段落可适度；code/pre 不可	破坏 Python/ASCII art/表格
Hyphenation repair	PDF postprocess	可基于行末/词典/模型	错误合并词
Header/footer removal	document structure postprocess	建议以 furniture/type 标注而非直接丢弃 raw	规则可能误删正文
Dehyphen + ligature	PDF normalization	记录转换	文本对齐/provenance 需要映射
URL canonicalization	metadata	用于 dedup/filter；正文另行决策	canonicalization 过强会合并不同资源

16. 推荐架构：Router → Candidates → Judge → Canonicalizer

Input sniffing: mime / source / size / language / native-text availability
Parser Router
Candidate parsers (fast / high-recall / structure-aware / OCR-VLM)
Diagnostics & Quality Judge
Select / merge / union by policy
Canonicalizer + provenance
Downstream cleaning / dedup / scoring

```
def parse(obj):
    profile = sniff(obj)                # mime, source, size, text-layer, layout hints
    routes = routing_policy(profile)
    candidates = [run_parser(p, obj) for p in routes]
    candidates = [c for c in candidates if c.status == "ok"]
    scored = [(c, parser_quality(c, profile)) for c in candidates]
    chosen = select_or_merge(scored, policy=profile.policy)
    return canonicalize(chosen, raw_ref=obj.ref, keep_diagnostics=True)
```

路由条件	Fast path	High-quality path	原因
普通英文新闻/博客	Trafilatura/Resiliparse	双 extractor + quality judge	主正文规则成熟

路由条件	Fast path	High-quality path	原因
含大量 table/code/math 的 HTML	结构感知 extractor	模型式 HTML parser	普通 extractor 可能破坏结构
单栏 born-digital PDF	native PDF text	Docling/MinerU fallback	降低 GPU 成本
复杂学术 PDF	GROBID/Docling	MinerU/VLM parser	需要 hierarchy/reading order/formula/table
扫描 PDF	OCR	VLM/高精 OCR	无 text layer
DOCX/XLSX/PPTX	native OOXML	document intelligence parser fallback	原生结构比渲染更丰富

17. Parser Quality Gate：在 Filtering 前先判断“有没有解析对”

Gate	示例指标	失败动作
Decode Gate	replacement-char ratio、decode exception	fallback charset / quarantine
Empty/Short Gate	visible chars、block count	尝试 alternative parser；标 parser_fail 而非 low_quality
Boilerplate Gate	link density、nav repetition、template ratio	换 main-content mode / second extractor
Structure Gate	heading/list/code/table count 与 DOM hints 一致性	结构 parser fallback
Reading-order Gate	column/layout confidence、line y-order anomaly	layout parser/VLM fallback
OCR Gate	OCR confidence、dictionary/LM score、character entropy	高精 OCR / reject page
Cost Gate	CPU/GPU seconds, bytes/sec	只对 high-value/uncertain 样本升级 parser
Stability Gate	timeout/OOM/crash	process guard、shard isolation、retry budget

不要混淆：“parser 失败”不是“文档低质量”。如果 parser failure 被当成 quality filter drop，数据团队会错误地把系统弱点解释为互联网内容质量问题，长期造成数据分布偏差。

18. 数据解析评测矩阵：从组件指标到模型指标

层级	指标	回答的问题	什么时候用
Byte/Decode	decode success、mojibake rate	字节是否正确进入 Unicode？	所有 source
Text	precision/recall、edit distance	正文文字是否正确？	HTML/PDF/OCR
Boilerplate	boilerplate leakage / main-content recall	菜单广告是否混入、正文是否被误删？	HTML
Structure	block F1、heading/list/code/table recall	结构保留得怎样？	HTML/PDF/Office
Order	reading-order accuracy/edit	块顺序是否正确？	PDF/Slides
Yield	token/char yield, unique yield	解析器实际提供多少训练材料？	大规模 pipeline
Robustness	fail/timeout/OOM rate	十亿级运行是否可靠？	production
Cost	\$/TB、CPU-sec/page、GPU-sec/page	能否覆盖训练规模？	production
Downstream	validation loss、early-signal benchmark	解析差异是否真的提升模型？	关键 parser 决策

FineWeb 的做法给出了最重要的经验：关键 parser 决策最终用小模型 ablation 验证，而不是只用人工观感；2026 多 extractor 研究也把 token yield 与下游任务联合考察。[1][2]

19. Parser Ablation：如何证明某个解析器真的更好

固定同一批 raw documents
Parser A vs Parser B（只改一个变量）

统一 normalization / filtering / dedup
抽样相同 token budget
训练相同小模型
比较 loss + broad benchmarks + parser-specific tasks
检查被新增/丢失文档的分布

控制变量	要求
Raw source	同一 crawl/shard/document IDs
Token budget	A/B 训练 token 数相同，避免“数据更多”伪装成“质量更好”
Model/optimizer	架构、参数、LR、batch、tokenizer 完全一致
Downstream pipeline	filter/dedup/packing 保持一致
Evaluation	通用任务 + 结构相关任务；HTML table/code 应单独测
Multiple seeds/slices	防止单次采样噪声
Cost accounting	记录 parsing CPU/GPU/IO，使收益可转为 ROI

最容易犯的实验错误：Parser B 产生更多 token，然后按“同一 epoch”训练，会让模型看到更多 token，从而无法区分 parser 质量提升和训练量提升。正确比较应固定训练 token 数和其他处理步骤。

20. 失败模式地图：解析错误如何一路污染到模型

解析错误	直接结果	下游数据问题	潜在模型表现
导航泄漏	Home/Login/Share 重复	重复率上升、有效 token 比下降	网页模板/菜单生成偏置
正文漏抽	文章只剩标题/摘要	知识覆盖下降	事实/长尾知识缺失
段落乱序	跨栏交错	语篇因果破坏	长文 coherence 变差
表格压平	行列关系消失	结构化事实错误	table QA/数字关系变差
代码空白压平	缩进丢失	代码语义破坏	Python 等生成错误
公式损伤	上下标/符号丢失	错误数学文本	数学/科学能力受损
OCR mojibake	字符随机错误	高 entropy 假 token	语言建模 loss 浪费
重复 header/footer	每页相同文本	文档内部重复	记忆模板、浪费上下文
错误语言解码	字符集乱码	lang ID/filter 误判	语言覆盖被系统性丢失
parser crash bias	某类页面总失败	source distribution 偏差	能力盲区难被发现

21. Production Blueprint：可扩展、可回放、可迭代的数据解析系统

组件	职责	推荐设计
Raw Object Store	保存原始 WARC/PDF/repo snapshot	immutable + content hash + source timestamp
Shard Manifest	把超大 corpus 切成可重跑 shard	document IDs + byte ranges + parser version
Sniffer	快速识别 mime/encoding/layout/text-layer	CPU cheap、先路由后重 parser
Parser Workers	执行 extractor/OCR/VLM	进程隔离、timeout、memory guard、batching
Candidate Store	保留多个 parser 结果	只对实验样本或难例保留全候选，控制存储
Canonicalizer	统一 schema 与可确定序列化	版本化；所有转换可记录
Diagnostics Store	统计 parser health	失败原因、耗时、yield、结构数量、语言/域名分布
Quality Judge	判断 parser output 是否可信	规则 + 小模型 + cross-parser agreement
A/B Registry	追踪 parser 实验	raw sample hash + config + downstream result
Lineage/Deletion	支持 source 删除与重建	raw_ref → canonical → filtered → token shard 全链路

22. 成本策略：Fast Path + Selective Escalation

高精度 parser 不必覆盖全部数据。现实最优解通常是“便宜 parser 处理大多数简单样本，只把高价值或不确定样本升级到昂贵 parser”。这与推理系统里的级联模型类似。

层级	Parser	触发条件	目标
L0	WET/简单 text extraction	极低预算、初筛	快速获得 baseline/统计
L1	Trafilatura/Resiliparse/native parsers	普通 HTML/Office/PDF text-layer	覆盖 80-95% 简单样本
L2	layout-aware parser / GROBID / Docling	结构丰富、学术、多栏	恢复 hierarchy/order/table
L3	VLM/OCR/MinerU 类	扫描、复杂公式表格、L2 低置信	最大化高价值难例质量
L4	multi-parser union/judge	训练 horizon 长、结构能力重要、难例	扩大 unique high-quality token pool

23. 工具/策略选择矩阵

场景	首选	备选	核心判断
Common Crawl 通用英文 Web	WARC + Trafilatura/Resiliparse	WET baseline	预算允许时，WARC 可控性明显更强
超大规模低预算 Web	Resiliparse/WET + strict monitoring	Trafilatura subset ablation	先拿 throughput，再对关键分布做高质量重解析
Web table/code/math 很关键	multi-extractor / structure-aware model parser	Trafilatura + DOM special handlers	plain text extractor 不是最终形态
科学论文	GROBID / Docling / MinerU	native text + rules	需要 citation/hierarchy/formula/table 决定路线
企业普通 PDF	Docling / MinerU	native + OCR cascade	先分 born-digital vs scanned
扫描/复杂 PDF	MinerU/VLM/OCR pipeline	multiple OCR	要测 reading order/table/formula，不只 OCR
DOCX/XLSX/PPTX	native OOXML	Docling/MinerU	不要默认 PDF 化
Code corpora	repo graph + language/license parser	file text only	repo/path/version/provenance 是一等信息

24. 可沉淀的 Know-How / Know-Why

原则	Know-why	Know-how
Parser 是数据超参数	解析改变训练分布	把 parser config/version 纳入 dataset experiment registry
保留 raw + provenance	未来 parser 会进步、合规需要回溯	raw immutable; canonical 中存 raw_ref/hash/offset
结构不要过早压平	结构一旦丢失无法后补	先 blocks/tree，最后才序列化为 text
Parser failure != low quality	否则会系统性偏置数据	独立 failure code；对失败样本统计 source/language/layout
固定 token 做 ablation	避免数量混淆质量	同 raw、同 filtering、同 token budget、同 model
多 parser 不等于全部全量跑	成本可能失控	routing + uncertainty escalation + experimental union
PDF 必须测 reading order	字符对但顺序错仍是坏数据	layout/reading-order metric 加入 gate
代码要保留 repo graph	跨文件关系是核心训练信号	path/revision/lang/license 保存在 metadata/serialization
Normalization 要可逆/可记录	误修改会污染文本	raw offsets 或 transform log；代码块特殊处理
最终指标是模型	局部 parser metric 不一定等于预训练收益	小模型 early-signal + 结构任务 + cost ROI

25. 一份可直接落地的 Parser Design Checklist

- 所有 raw object 是否有稳定 ID、hash、source、timestamp 和可回放位置？
- 是否将 parser failure 与 quality filter drop 区分？
- HTML 是否有 WARC/WET 成本对照实验，而不是凭经验选择？
- 是否保留 heading/list/code/table/formula 等结构块？
- 是否对普通页面和复杂页面做 parser routing？
- 是否统计 decode error、empty ratio、boilerplate ratio、token yield、timeout/OOM？
- PDF 是否区分 native/scanned/hybrid，并评估 reading order？
- PDF table/formula 是否有专门指标，而不只看 OCR？
- Office 文档是否优先 native parser？
- Code 是否保留 repo/path/revision/language/license？
- 是否固定 raw source、后续 filtering 和 token budget 做 parser A/B？
- 是否至少有一个“最终模型指标”裁决 parser 选择？
- parser 版本升级时能否只重跑受影响 shard，而不是重建全数据湖？
- 是否有 cross-parser disagreement 样本池用于持续改进？
- 最终 serialization 是否可从 canonical blocks 确定地重建？

结论：LLM 数据解析层的演进方向

第一阶段：WARC/PDF/Repo → plain text：先解决“能不能把数据拿出来”。

第二阶段：main-content / layout-aware parsing：解决“拿出来的是不是正确正文”。

第三阶段：structure-preserving canonical document：解决“标题、代码、表格、公式、reading order 是否保留”。

第四阶段：multi-parser routing + model-based parser + eval-driven ablation：把 parser 变成可优化的数据配方变量。

对预训练工程而言，最重要的认知是：Filtering 决定“什么内容留下”，而 Parsing 决定“系统看见的内容究竟是什么”。如果解析层已经把正文删掉、把代码压平、把表格打乱、把多栏顺序错置，后续再先进的 dedup、quality classifier 和 tokenizer 都无法恢复这些信息。

参考资料（优先一手论文 / 官方文档）

[1] Hugging Face, FineWeb: decanting the web for the finest text data at scale, 2024. FineWeb official long-form report and dataset documentation.

[2] Li et al., Beyond a Single Extractor: Re-thinking HTML-to-Text Extraction for LLM Pretraining, arXiv:2602.19548, 2026.

[3] Common Crawl, Get Started / WARC-WAT-WET format documentation.

[4] Common Crawl, June 2026 Crawl Archive Now Available (CC-MAIN-2026-25), 2026.

[5] ChatNoir Resiliparse documentation: FastWARC, HTML parsing, HTML2Text/main-content extraction.

[6] OpenDataLab, MinerU official documentation; MinerU project introduction and format support.

[7] IBM / Docling project, Docling Technical Report and DoclingDocument documentation.

[8] GROBID official documentation, scientific PDF to structured TEI XML.

[9] Allen Institute for AI, Dolma Toolkit data format / dataset documentation.

[10] WHATWG, Encoding Living Standard, updated 2026.

[11] Unicode Consortium, Unicode Standard Annex #15: Unicode Normalization Forms.

- [12] Trafilatura official documentation and ACL-IJCNLP 2021 system demonstration paper.
- [13] Ma et al., AICC: Parse HTML Finer, Make Models Better - A 7.3T AI-Ready Corpus Built by a Model-Based HTML Parser, arXiv:2511.16397, 2025.
- [14] Niu et al., MinerU2.5: A Decoupled Vision-Language Model for Efficient High-Resolution Document Parsing, 2025; Wang et al., MinerU2.5-Pro, 2026.
- [15] Allen Institute for AI, peS2o: Pretraining Efficiently on S2ORC, dataset processing documentation.
- [16] OpenDataLab, OmniDocBench official benchmark / arXiv paper; text, table, formula, layout and reading-order evaluation.
- [17] Allen Institute for AI, S2ORC / s2orc-doc2json: PDF2JSON, TEX2JSON, JATS2JSON pipeline.
- [18] Lozhkov et al., StarCoder 2 and The Stack v2: The Next Generation, 2024; repository reconstruction and code corpus processing.

研究说明

本文将公开论文与官方文档中的事实性结论，与面向生产系统的工程归纳分开处理：带编号 [n] 的事实对应参考资料；架构建议、schema、routing/checklist 属于基于这些证据的工程综合与推导。具体阈值和 parser 选择应在你的目标语种、网页类型、模型规模与训练 horizon 上重新做 ablation。